Project Phoenix: System Architecture Document

By Anoop VK 7 min 14 Add a reaction Karma Page Builder Request approval
Ask to AI ChatGPT Assistant Show less

Version: 2.0 **Date:** December 18, 2025 **Owner:** Cloud Engineering & Security (CISO Office) **Status:** Approved for Implementation

1. Executive Summary

Project Phoenix is a cloud-native, serverless-first platform designed to deliver high scalability, zero-trust security, and cost-efficiency. The architecture utilizes **AWS Lambda** for compute, **Amazon CloudFront** for global content delivery, and a segregated network design (VPC) to protect data persistence layers. This document outlines the logical and physical architecture of the system as depicted in the v2 design.

2. High-Level Architecture Diagram

The following diagram illustrates the current architecture, highlighting the separation of concerns between Platform and App users, the edge security layer, and the private compute logic.

Application Architecture

AWS Deployment Diagram

3. Architecture Component Breakdown

Based on the v2 diagram, the system is divided into four primary layers: **Edge/Entry**, **Frontend Serving**, **Compute/Backend**, and **Data/Persistence**.

3.1 Edge & Entry Layer (Public Zone)

Traffic enters the system through a secure, globally distributed edge network.

- **Amazon Route 53:** Manages DNS resolution, routing Platform and App users to the nearest CloudFront entry points.
- **AWS WAF (Web Application Firewall):** Attached to CloudFront distributions to provide Layer 7 protection against SQL injection, XSS, and DDoS attacks.
- **Amazon Cognito:** Manages authentication and user pools. It provides the identity tokens (JWT) required for users to access the API Gateway.

3.2 Frontend Delivery (Content Delivery Network)

The architecture separates traffic for "Platform Users" and "App Users" to ensure granular control over caching and security policies.

- **Platform CloudFront CDN:** Serves the administrative interface. It routes static assets from the `ui-platform` S3 bucket and dynamic API calls to the API Gateway.
- **App CloudFront CDN:** Serves the end-user application. It routes static assets from the `ui-renderer` S3 bucket and API calls to the API Gateway.

3.3 Compute & Application Layer (VPC Private Subnet)

All business logic resides within a Virtual Private Cloud (VPC) to ensure isolation.

- **Amazon API Gateway:** The central entry point for all REST/WebSocket requests. It acts as the "Front Door" for the backend, enforcing throttling and validation before passing requests to Lambda.
- **App-Manager Lambda:** The core compute unit running in a **Private Subnet**. This serverless function executes business logic, processing requests from the API Gateway. It is isolated from the public internet for enhanced security.
- **VPC Interface Endpoints:** Used to securely connect the Lambda function to AWS managed services (like Secrets Manager and CloudWatch) without traffic traversing the public internet.

3.4 Data & Persistence Layer (Private Subnet)

Data stores are strictly isolated within the Private Subnet, accessible only by the `app-manager` Lambda.

- **Amazon RDS:** The primary relational database (Aurora Serverless) for transactional data.
- **Amazon ElastiCache:** Provides in-memory caching (Redis) to reduce database load and improve latency.
- **MongoDB Atlas:** The NoSQL store for unstructured data, connected securely via VPC Endpoints/Peering.
- **Jiffy-Drive (S3):** Used for storing large objects and user files, accessed via an **S3 Gateway Endpoint** to keep traffic within the AWS network.

3.5 Networking & Egress

- **Public Subnet & NAT Gateway:** A NAT Gateway is placed in the Public Subnet to allow the Lambda functions in the Private Subnet to access the internet (if required for 3rd party APIs) while preventing inbound connections.
- **Private Subnet:** Hosts the Lambda, RDS, and ElastiCache, ensuring no direct public access.

4. Security & Observability Stack

4.1 Security & Governance

- **AWS Secrets Manager:** Securely stores database credentials and API keys. The Lambda function retrieves these at runtime, ensuring no secrets are hardcoded.
- **Amazon GuardDuty:** Provides intelligent threat detection by monitoring VPC flow logs and DNS logs for malicious activity.
- **AWS Security Hub:** Aggregates security alerts and compliance status into a single dashboard.
- **AWS AppConfig:** Manages dynamic configuration flags, allowing feature toggles without redeploying code.

4.2 DevOps & Monitoring

- **Amazon ECR:** Stores the container images for the Lambda functions.
- **AWS CodeArtifact:** Manages software packages (Maven/npm) to prevent dependency chain attacks.
- **Amazon CloudWatch:** Collects logs and metrics from all services (Lambda, API Gateway, RDS) for monitoring and alerting.
- **AWS X-Ray:** Provides distributed tracing to visualize requests traveling from CloudFront -> API Gateway -> Lambda -> Database, aiding in debugging latency issues.

5. Design Decisions & Rationale

1. **Dual CloudFront Distributions:** Separating `Platform` and `App` users allows for distinct caching policies and WAF rulesets tailored to internal vs. external traffic profiles.
2. **Lambda in VPC:** Placing Lambda in the VPC is necessary to access RDS and ElastiCache securely. The **VPC Interface Endpoints** reduce latency and cost by keeping AWS service traffic internal.
3. **S3 Gateway Endpoint:** Utilizing a Gateway Endpoint for S3 (`Jiffy-Drive`) avoids data processing charges associated with NAT Gateways and improves S3 transfer speeds.